Personalized Offline-Capable AI Tutoring Agent for Rural Education

Myo Thu Htet, 24249488, Author, hmy5499@autuni.ac.nz
Dr. Kenneth Johnson, Supervisor, Senior Lecturer, kenneth.johnson@aut.ac.nz

Auckland University of Technology, Auckland, New Zealand

Abstract. The educational opportunities for rural students remain restricted because they lack reliable internet access and sufficient educational resources. The operation of cloud-based AI tutoring systems depends on continuous internet access and powerful processing capabilities which prevents their use in distant locations. The research creates a Personalized Offline-Capable AI Tutoring Agent which operates on a **Raspberry Pi 5** device that functions as both a wireless hotspot and an AI processing server to support subject-specific learning through a **Flask**-based web interface and **llama.cpp quantized LLM** integration. The experiments prove that the system operates in real-time with low latency while using minimal energy to deliver tutoring services which confirms its potential as an affordable offline AI tutor for rural educational settings.

Keywords: Offline AI Tutor, AI Tutoring Agent, Offline Learning, Raspberry Pi, Rural Education, Large Language Models, Edge Computing

1 Introduction and Background

1.1 Introduction

The development of society and economy relies on education but millions of students from rural and conflict-affected regions encounter major challenges to obtain quality learning opportunities. Artificial Intelligence (AI) has experienced rapid growth and educational institutions across developed countries and urban areas have started adopting it during recent years. AI-driven education provides customized learning experiences through automated tutoring systems and data-driven educational assistance. AI-driven educational solutions have primarily focused on urban areas and technologically advanced countries which leaves behind marginalized populations in remote and unstable regions.

The research aims to develop a Personalized Offline-Capable AI Tutoring Agent which serves resource-constrained areas such as rural Myanmar. The Raspberry Pi 5 functions as both Access Point and server to run the AI Tutoring system which allows real-time interactive learning without requiring internet access or expensive infrastructure. This research aims to create an operational way for technology access equality in AI education through experiments with light-weight Large Language Models (LLMs).

1.2 Background and Motivation

The educational system in Myanmar rural areas and conflict-affected communities experiences major disruptions. The delivery of quality education faces barriers because of insufficient infrastructure combined with unstable electricity supply, limited digital resources and forced population displacement due to armed conflicts. Students face educational deprivation because of unstable teaching staff and limited textbook and internet availability which creates a larger educational gap compared to urban students and students from stable regions.

Other countries and cities use AI-powered learning platforms to enhance classroom participation while delivering personalized education. Mainstream education institutions across countries with strong infrastructure now use intelligent tutoring systems together with adaptive curriculum and AI-driven assessments. The fast worldwide adoption of AI education systems creates educational inequalities because students who access advanced digital learning systems gain advantages yet students in conflict zones and rural areas face growing marginalization because of their restricted access.

The research motivation arises from the urgent need to bridge technological and educational gaps. The project seeks to provide rural students with independent AI tutor access to knowledge through a system that functions without internet or expensive cloud infrastructure. The system provides an answer to keep education going for Myanmar students in conflict zones and similar situations during crisis situations. The research promotes global education democratization through AI by using technology to help disadvantaged groups instead of treating it as an exclusive privilege.

2 Literature Review

2.1 AI Tutoring in Education

The recent progress in natural language processing (NLP) has shaped interest in implementing large language models (LLMs) as educational resources. (Alhafni et al., 2024) [1] provide a detailed tutorial about LLM applications for reading, writing, speaking and intelligent tutoring systems (ITS) while discussing their benefits and drawbacks including resource needs, biases and hallucinations issues. (Wen et al., 2024) [2] suggest that LLMs can create personalized learning experiences using adaptive learning systems but they need strong algorithms, user-friendly interfaces and ethical protection mechanism. The research of (Choi et al., 2023) [3] analyze GenAI chatbot that operates in 122 Sierra Leonean schools and teachers use the system for planning lessons and classroom management thus showing LLM tools can help resource-limited education environment. Research indicates that students depend on LLMs for help but they lack understanding about the system's operational limitations.

The study by (Denny et al., 2024) [4] shows students want AI teaching assistants for immediate feedback but they want guidance instead of complete answers to maintain their independence. The research reveals that more than 70 percent

of secondary students currently use LLMs for their schoolwork across different subjects. Even though AI hallucinates and incorrect reasoning, they still prefer to use AI systems for their specific needs and customized learning environments (Zhu et al., 2024) [5]. The research highlights that mainstream education experience both positive and negative attitudes toward LLMs because offline tutoring systems need to develop educational interactions that follow proper teaching methods while helping students develop self-regulation skills.

New frameworks attempt to enhance LLMs through the addition of reasoning capabilities and agent-like functionality. (Kamalov et al., 2025) [6] explain how agentic workflows which include reflection, planning , tool usage and multi-agent teamwork enhance the reliability and transparency of educational agents. (Ferrag et al., 2025) [7] examine evaluation criteria for LLM reasoning systems and autonomous agents while recommending future research on sophisticated reasoning methods and system failure detection and security measures. (Jaldi et al., 2024) [8] recommend using hybrid architectures that combine neural and symbolic components to enhance better explainability through more detailed assessment of student understanding. The LLM-based teachable agent AlgoBo (Jin et al., 2024) [9] uses structured prompts to help students teach the system while experimental results show that new users create detailed educational dialogues and find the system interactive. To have effective learning experiences, language-generation features must be integrated with structured educational procedures in developing personalized tutoring agents.

2.2 AI Tutoring in Rural Areas

AI tutoring expansion into rural areas needs solutions for both infrastructure limitations and sociocultural barriers. The intelligent learning framework developed by (Bali, 2025) [10] uses reinforcement learning and real-time learner modeling to create content adaptations for developing country students through simulations with 1,000 synthetic learners who achieve better learning results and dropout risk remains under 4 %. The research by (Karimy et al., 2024) [11] examines AI-based educational systems in Afghanistan which operate on basic hardware with restricted power supply and network access. The research shows that NGOs can establish AI-based educational platforms despite Internet access and device availability challenges. The conference reports from (Karimy et al., 2024) [12] demonstrate that AI-powered online platforms can transcend geographical boundaries yet require solutions to overcome digital inequalities between genders. The solution to these inequalities requires developers to create content that respects local cultures while enabling offline access and supporting low-bandwidth connections.

The implementation of small language models (SLMs) on edge devices represents a viable solution. (Ghorbani & Fattahi, 2025) [13] demonstrate how SLMs function as virtual tutors for underdeveloped areas through their ability to operate independently on low-power devices while supporting native language interactions. The authors explain that power outages in 80 % of schools require power-efficient solutions which SLMs can help solve while reducing gaps between

genders and resources. (Trajkovski & Hayes, 2025) [14] suggest AI-assisted assessment systems which use learner data to adjust content difficulty and provide specific feedback and researchers need to solve problems related to equity and bias and system transparency. The combination of personalized learning platforms with intelligent tutoring systems in rural schools leads to better student results in math and science while helping teachers manage their work more efficiently according to (Leong, 2024) [15]. The frameworks show that AI tutors for offline use need customization for specific areas through reinforcement learning and small models to create fair educational experiences.

2.3 LLM optimization techniques on Edge devices

The deployment of AI tutors as offline systems requires effective low-power hardware-based inference operations. (Lin et al., 2024) [16] developed activation-aware weight quantization (AWQ) that detects and adjusts 1% of essential weight channels through activation distribution analysis to support low-bit quantization without requiring back-propagation. (Yi et al., 2025) [17] introduce EdgeMoE as an on-device inference engine for mixture-of-experts models during 2024. The system maintains non-expert weights in device memory while it retrieves expert weights from external storage through bit-width adaptation and preloading to minimize I/O delays.

The research by (Ardakani et al., 2025) [18] shows how quantization-based optimization works on Raspberry Pi systems and shows that 2-bit and 4-bit quantization leads to better performance and energy efficiency because Llama1B running on Raspberry Pi reaches 14.78 tokens per second when quantized which represents a four times improvement over full precision. The research on single-board computers shows that local system model execution needs additional power but API-based models produce superior results by using less system resources when they connect to the internet (Sakai et al., 2024) [19].The results demonstrate that offline autonomy creates an energy efficiency trade-off.

Recent benchmarks of generative models on edge devices report that models achieve 73.3–85.9 % quality, produce 1.83–3.51 tokens/s and consume 0.93–1.76 energy units per token. Research studies between remote and on-device generation show remote fetching uses 3.5–8.9 times less energy than on-device generation while execution time directly affects energy consumption according to (Nguyen et al., 2025) [20]. The development of offline tutoring agents needs suitable model selection and quantization methods and hybrid systems that perform local operations and move work to accessible networks.

2.4 Offline infrastructure on Raspberry Pi

Delivering AI tutoring offline also entails reliable local networking and storage. (Islam et al., 2018) [21] expand their access-point configuration algorithms to manage client join/leave events and demonstrate their performance through

simulations and Raspberry Pi testbeds. (Cheah et al., 2015) [22] show that Raspberry Pi functions as an affordable server and wireless gateway for file transfer and IoT applications through XBee communication in 2015. (Slavin et al., 2024) [23] The Wi-Fi chip in Raspberry Pi access points restricts the system to support eight simultaneous clients. The study demonstrates the requirement for precise capacity planning because of this limitation. The Raspberry Pi zero W functions as a media server to deliver offline content which supports wireless access for 50 users according to (Jadhav & Malode, 2019) [24]. The VPN travel router enables secure remote server access to devices that use available networks as (Pierson, 2016) [25] explain yet users need to evaluate both power usage and system complexity.

3 Research Questions and Scoping

3.1 Research Question

The architecture and deployment of offline AI Tutoring agent on edge devices have some unique challenges compared to cloud-based AI tutoring agent. Unlike extensive computing power and digital resources cloud infrastructure, localized offline system always prioritize efficiency, accessibility and usability within hardware constraints. To address these challenges, this research focuses on the following questions.

System Design and Accessibility

1. How can an AI tutoring agent and LLM be designed to function effectively in offline, low-resource environments using affordable hardware such as the Raspberry Pi 5?

Performance and Resource Constraints on Low-Resource Devices

2. What are the limitations of running LLMs on Raspberry Pi in terms of response throughput by concurrent users and requests?

3.2 Aim

The aim of this research is to investigate how to design AI Tutoring agent effectively and deployed on offline and low cost hardware such as Raspberry Pi 5. Specifically, this study tries to explore two key aspect. The first one is that architecture design and accessibility of AI tutoring agent that can deliver impactful educational assistance without internet. The second is the performance and resource constraints of running LLM on edge devices. By doing these experimentation, this research focus to provide feasibly , limitations and potential of offline AI tutoring agents on rural areas.

3.3 Scope of the Study

This experiment is limited to the design and evaluation of an offline AI tutoring agent deployed on a Raspberry Pi 5 as primary low-cost computing infrastructure and the scope only focus on two main aspects.

– System Design and Accessibility - To develop and test AI system using optimized LLM locally without internet.
– Performance and Resource Constraints - To measure performance metrics such as response throughput by concurrent users and request.

The study doesn't include field experiments or survey with students and teachers in rural area due to feasibility and accessibility constraints. The evaluation only conduct on prototype simulation and controlled testing environment. The research doesn't include multilingual functionality, especially support for Myanmar language because of limitation and hallucinations on current available optimized LLMs. The study only concentrates on feasibility and identifying limitations on offline AI tutoring system on low-resource environment.

3.4 Contributions of the study

This research contribute to both academic and practical applications in AI for education. Proposing system design for offline AI Tutoring agent using Raspberry Pi 5 and demonstrating affordable hardware can be used for AI support in education without internet connectivity. Performance Benchmarking for LLM on pi, limitations and feasibility insights. Equality to access useful and impactful AI system in education and creating ground work for future studies.

4 Research Method

4.1 Research Plan

The research uses an experimental research method to evaluate feasibility of different LLM for personalized offline-capable AI tutoring agent on a Raspberry Pi 5 in rural education. The plans are the following:

Literature Review and Requirement Analysis

– Identify the challenges and limitation faced by rural students and lecturers especially restricted access to digital resources because of limited or expensive internet connectivity and resource constraints.
– Review existing literature and gaps in current existing solutions, usability and feasibility on offline AI tutoring, large language models (LLM) and Edge devices (eg. Raspberry Pi).

System Architecture and Prototyping

– Design the architecture of tutoring system including hardware setup and software stacks.
– Prepare to configure Raspberry Pi as both access point and tutoring server
– Setup External USB WiFi Adapter to extend coverage and stable connectivity.

– Implement Lightweight web ui interface and setup LLM inference engine.
– Research and choose LLM model for edge devices especially quantized models.
– Developed Agent Mode with functional tools to eliminate hallucination and support educational functionalities.

Deployment and Demonstration

– Setup controlled environment simulation rural conditions and pilot the system.
– Demonstrate system features and functionalities.

Testing, Evaluation and Iteration

– Benchmark model performance such as response throughput by concurrent users and requests.

Analysis and Reporting

– Analyze findings and data to access effectiveness of AI on Edge Device.
– Investigate and Report the limitations and opportunities for scaling the solution and its features.

4.2 Research Prototype Development

The proposed system is designed as a Personalized Offline-Capable AI Tutoring Agent operates on a Raspberry Pi 5 (16 GB). The system uses lightweight Large Language Models (LLMs) and agent mode to provide interactive learning in rural areas without internet connectivity.

The following components for the system:

Hardware Setup

– **Student Devices**: phone, laptop, pc to connect to AI tutoring system
– **Raspberry Pi 5** (16 GB RAM): operates as the central computing unit, web server and access point
– **TP-Link TL-WN722N** external Wi-Fi adapter: enhances the range of the pi access point hotspot network while maintaining stable connections for student devices.
– **Battery** or **Power Bank**: To provide power to Raspberry Pi.

(Refer to Figures: Figure 1)

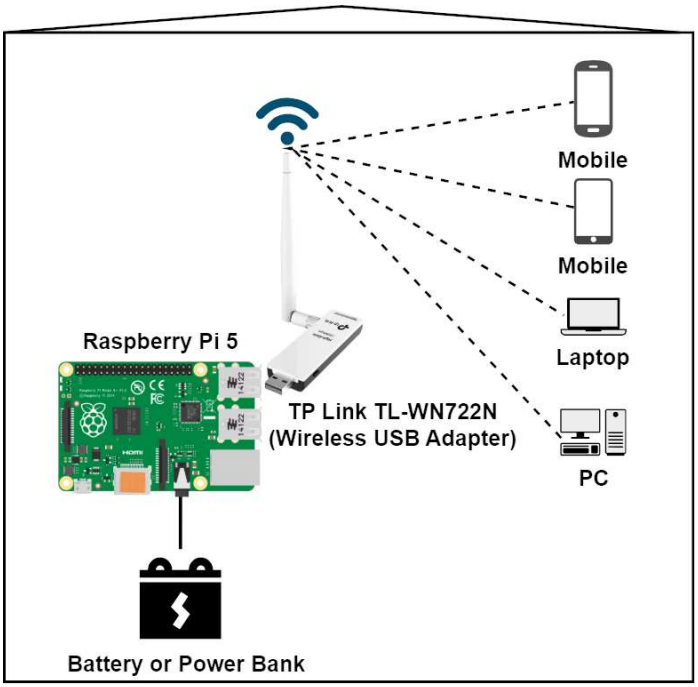


Fig. 1: Hardware Setup Diagram for AI Tutoring Agent

Application and Model

- **LLM Model**: will use Tinyllama-1.1b, Qwen2.5-3B, Mistral-7B and Meta-Llama-3-8B quantized version
- **Web UI** Interface: A lightweight customized web UI (python, html, css and js) via browser and can interact with AI Tutor.
- **llama.cpp** Runtime: is LLM inference engine and efficient inference backend that runs quantized LLM models on the Raspberry Pi 5, ensuring low memory usage and offline functionality
- **create_ap** Hotspot Manager: Scripted Wi-Fi access point service enabling Raspberry Pi to act as a local Access Point.
- **Browsers**: Chrome Browser to access Web UI.

System Architecture

- **Backend Agent Server**: ***agent_ server.py*** is implemented in Python using Flask (a micro python web framework). There are default system prompt for Agent mode, default system prompt tailoring for each subjects (Math, Coding, Science, English), Tools creation and execution in ***agent_ server.py***. It calls llama.cpp api to communicate with LLM.

– **Configuration**: ***config.json*** is a JSON-based config file and which will used by ***agent_ server.py***. Agent Mode system prompt and system prompt tailoring for each subjects can be configure in this config file. New Subject and specific behavior prompt can also be added in this file.
– **Frontend User Interface** : Browser based interface and implemented with HTML, CSS and JavaScript.
  - ***index.html*** provides the structure of chat panel with subject tabs.
  - ***styles.css*** manages responsive design, code-block formatting, chat bubble and icons.
  - ***script.js*** handles message rendering, api call and event triggers.
– **Deployment**: AI Tutoring system is hosted on Raspberry Pi 5 and Pi is also act as a local hotspot server. Students can connect to Pi Access Point Wifi and access AI Tutoring agent in a browser without internet.
– **Load Testing**:llm_load_test.py is a load and performance testing script designed to simulate multiple concurrent users and requests from the client side to evaluate the efficiency of LLM models on the Raspberry Pi.
– **Features**:
  - Subject based tutoring with tailored prompts.
  - Agent Mode for tool-augmented reasoning.
  - Offline accessibility and Lightweight

(Refer to Figures: Figure 2)

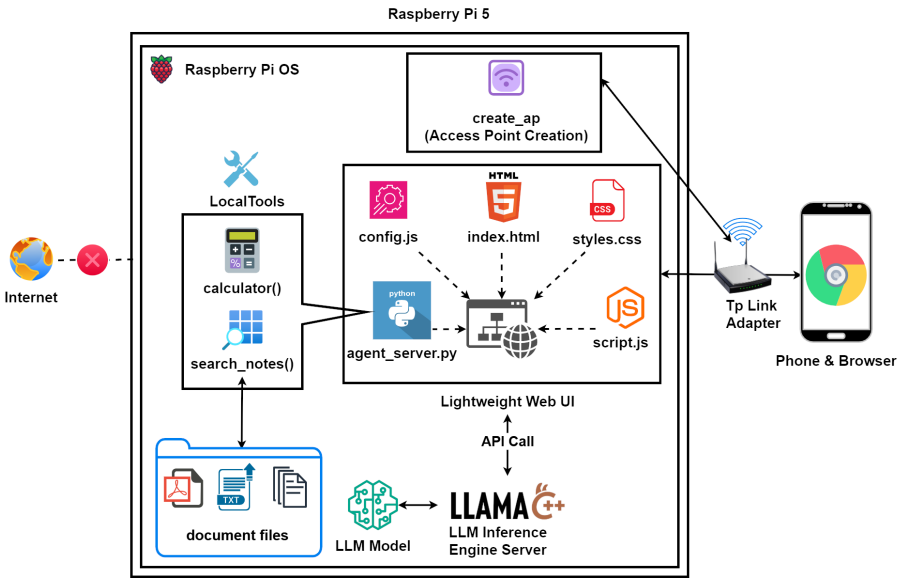


Fig. 2: System Architecture of AI Tutoring Agent

Demonstration

– **Access Point and AI Tutor Web UI QR**: There are 2 shell scripts, ***tpLink.sh*** and ***agent_ui.sh*** (please refer to appendix and github). ***tpLink.sh*** terminates all current WiFi connection, creates hotspot using ***create_ap*** tools, provides WiFi information and generates QR to connect to Pi's Access Point. ***agent_ui.sh*** launches llama.cpp, hosts lightweigh customized web ui, provides url links for both and generates QR for AI tutor web ui. (Refer to Figures: Figure 3 and 4)



Fig. 3: Hardware Setup for AI Tutoring Agent

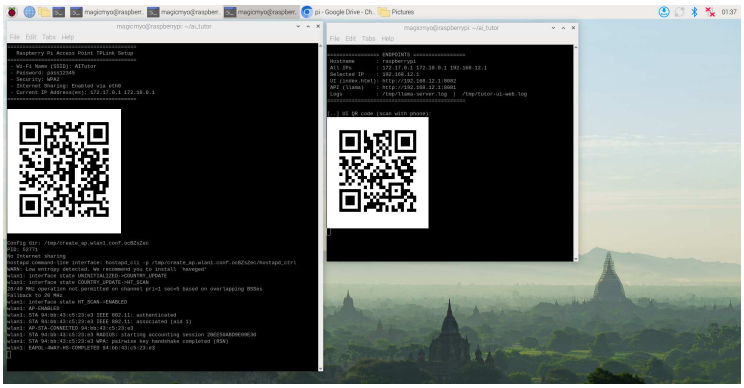
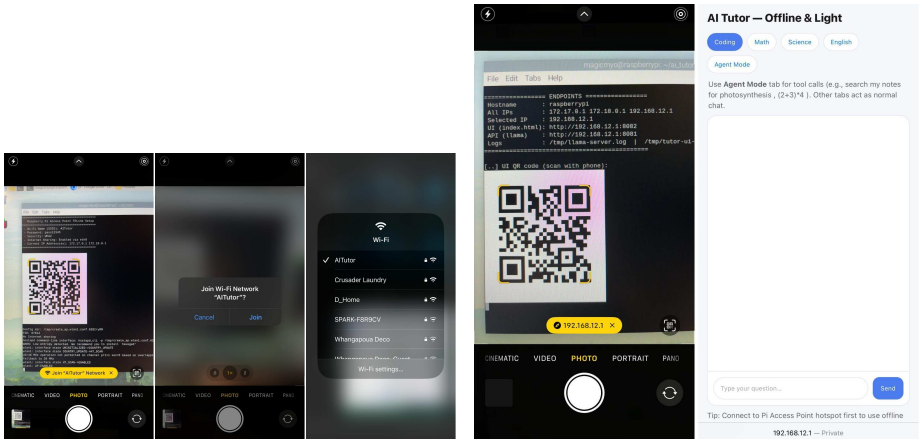


Fig. 4: Pi Access Point and AI Tutor Web Interface QR Code

– **Connecting to Pi's Access Point and AI Tutor Web UI**: By scanning QR or using information from ***tpLink.sh***, students can connect to Pi's Access Point with their devices. After connected to Pi's Hotspot, they can scan QR or typed url link on browser (eg. Chrome) from ***agent_ui.sh*** to connect to AI tutor web ui. (Refer to Figures: Figure 5)



Connecting to Pi Access Point Connecting to AI Tutor Web Interface

Fig. 5: Connecting to Pi Access Point and AI Tutor Web Interface

– **Using AI Tutoring Agent**: Now students can chat to AI tutor on their devices. There are subject tabs and Agent Mode. They can choose those tabs and get guidance as study assistant for that subject. (Refer to Figures: Figure 6)

– **Subject Tabs**: There are subject tabs (Coding, Math, Science and English).If student chooses Coding tab, tailored role playing system prompt will act as a coding tutor and will answer coding related questions. (Refer to Figures: Figure 6)

– **Agent Mode Tab**: Agent Mode can call tools and there are two tools have been created as a prototype, **calculator()** and **search_notes()**. It can make decision which tools to call and answer accordingly from tools output. **calculator()** is to calculate numeric operation without using LLM's own output to avoid hallucinations. **search_notes()** can search text from notes, lectures, books of local folder and provide short description. If tools have been used, there will be tool icon and tool name after output. (Refer to Figures: Figure 6)

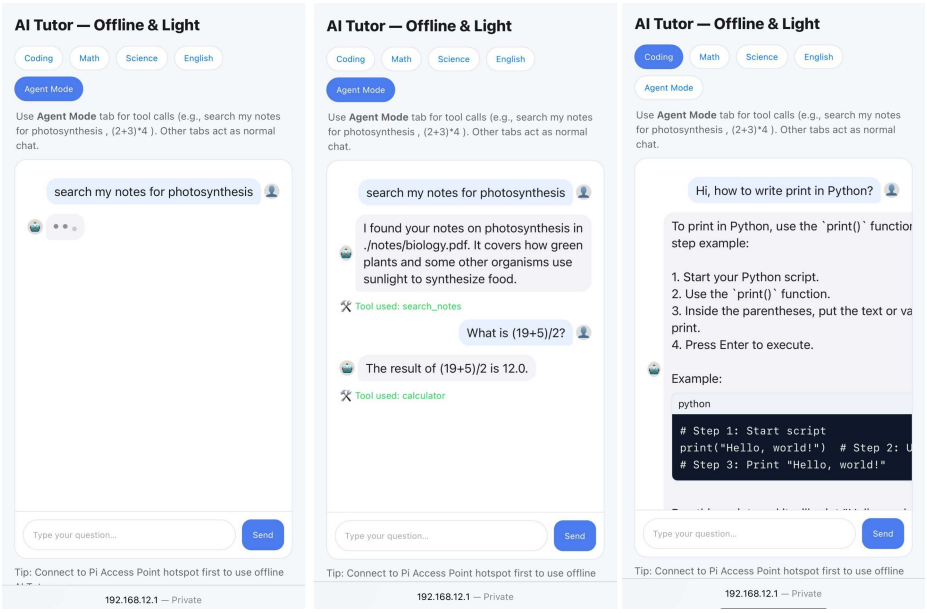


Fig. 6: Communicating with AI Tutor Agent

The complete demo implementation and scripts are accessible at GitHub Repository: https://github.com/magicmyo/offline_ai_tutor

5 Findings, Analysis and Evaluation

5.1 Finding

The Raspberry Pi 5 (16GB) ran the offline AI tutoring agent through testing four locally hosted LLMs (**Meta-Llama-3-8B, Mistral-7B, Qwen2.5-3B and TinyLlama-1.1B**) using a predefined load test setup. The models processed the following prompt which contained the same text for all models: **"Explain Newton's second law in a paragraph suitable for high-school students."**

The API timeout was established at 60 seconds while the system handled 2 and 4 concurrent users who submitted 4, 8 and 12 requests during each session.

The key findings are summarized below:

– The **Meta-Llama-3-8B** system together with **Mistral-7B** failed to process any request (100% failure) when two users and four users were active. The models failed to generate any valid output because their high computational requirements and memory usage exceeded the 60-second time restriction. (Refer to Figures: Figure 7,8,9 and 10)

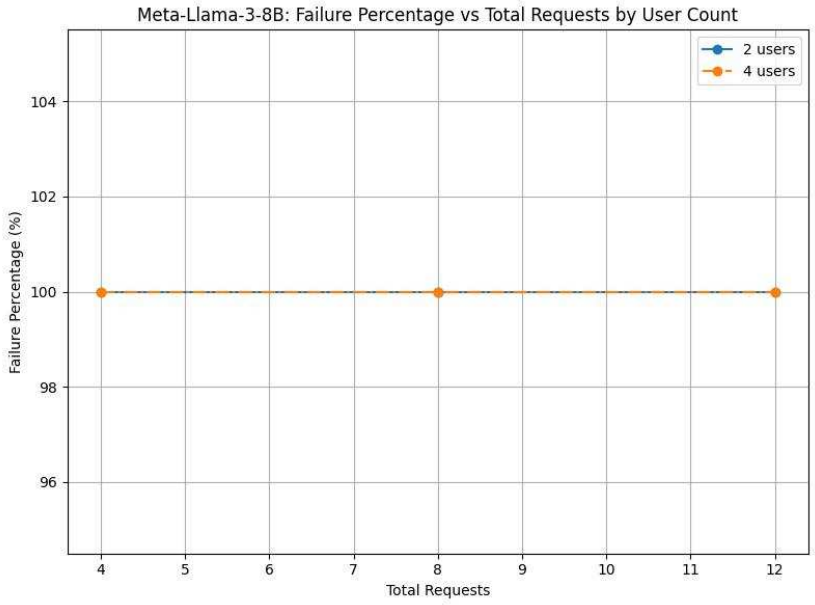


Fig. 7: Meta-Llama-3-8B Failure vs Requests

– **Qwen2.5-3B** achieved partial success, with failure rates of approximately 25% for 2 users and increasing to 50–67% for 4 users. The throughput grad-

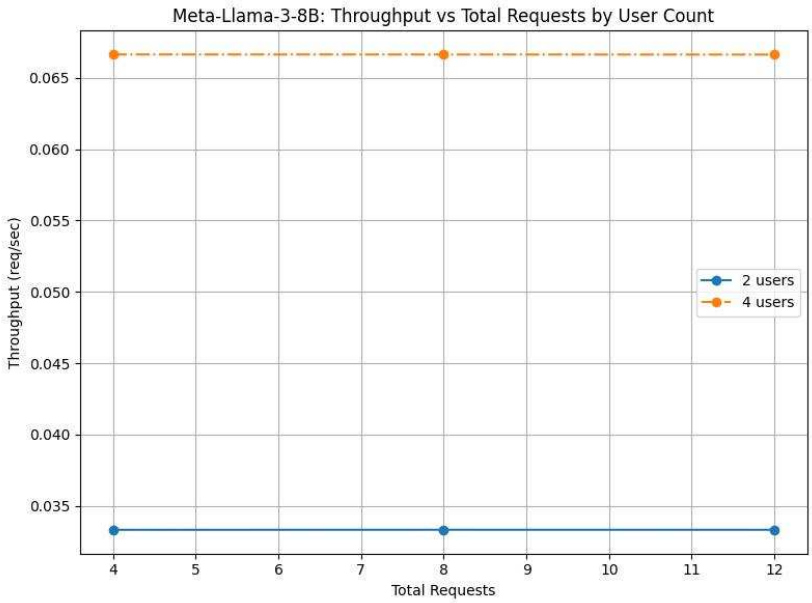


Fig. 8: Meta-Llama-3-8B Throughput vs Requests

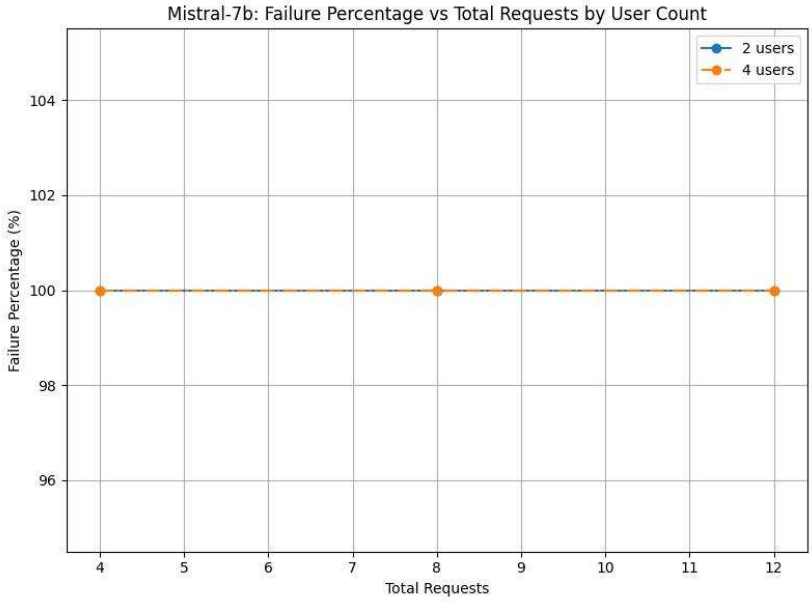


Fig. 9: Mistral-7B Failure vs Requests

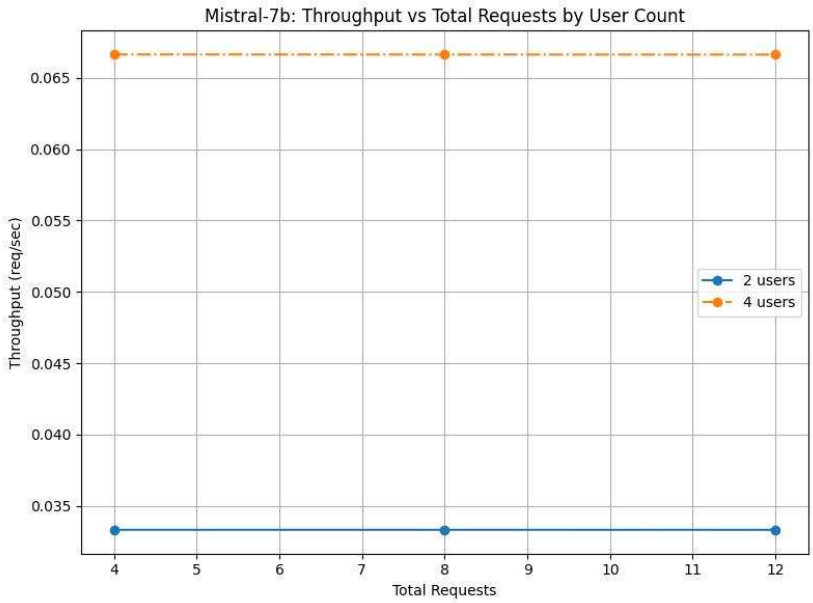


Fig. 10: Mistral-7B Throughput vs Requests

ually improved from 0.04 req/sec to 0.07 req/sec, indicating moderate handling capacity under limited concurrency. (Refer to Figures: Figure 11 and 12)

- The **TinyLlama-1.1B** model achieved the highest stability and scalability because it maintained a low failure rate of 9–25% when two users were active and its failure rate increased to 50–67% when four users were online. The system achieved its peak performance at 0.07 req/sec during this test case. (Refer to Figures: Figure 13 and 14)
- The comparison charts demonstrate that TinyLlama and Qwen2.5 processed requests better than Meta-Llama and Mistral which failed to operate under any system load.

5.2 Analysis

The research findings show how system dependability changes when using local LLMs for education based on model size and processing speed.

- **Failure Trend**: Both **Meta-Llama-3-8B** and **Mistral-7B** produced 100% timeouts, confirming that models above 7B parameters are unsuitable for Raspberry Pi 5 deployment under real-time tutoring conditions. (Refer to Figures: Figure 15)
- **Throughput Trend**: The system processed more requests when **Qwen2.5-3B** and **TinyLlama-1.1B** handled multiple users at once because it used

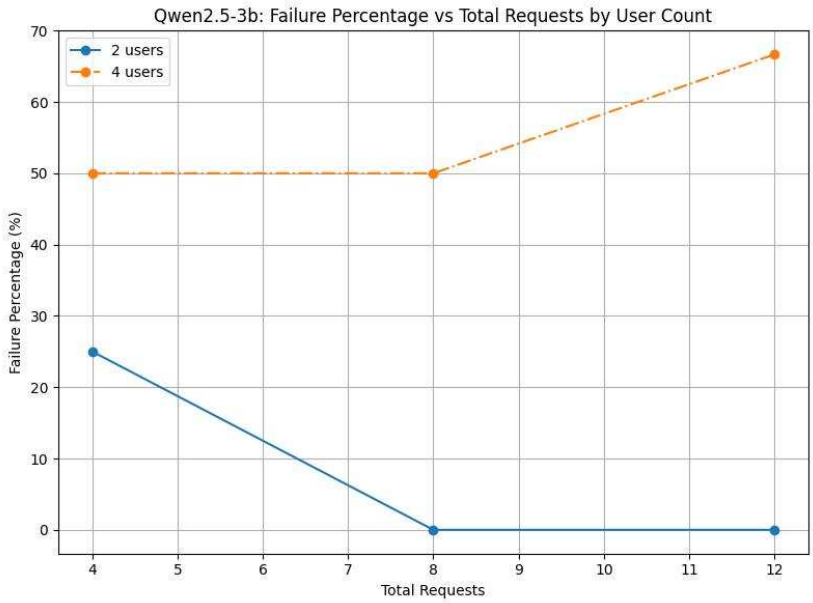


Fig. 11: Qwen2.5-3B Failure vs Requests

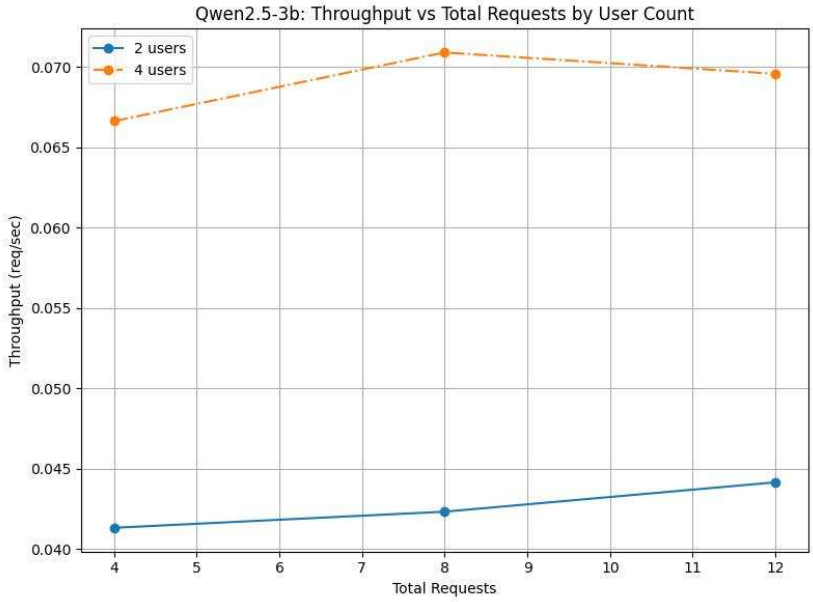


Fig. 12: Qwen2.5-3B Throughput vs Requests

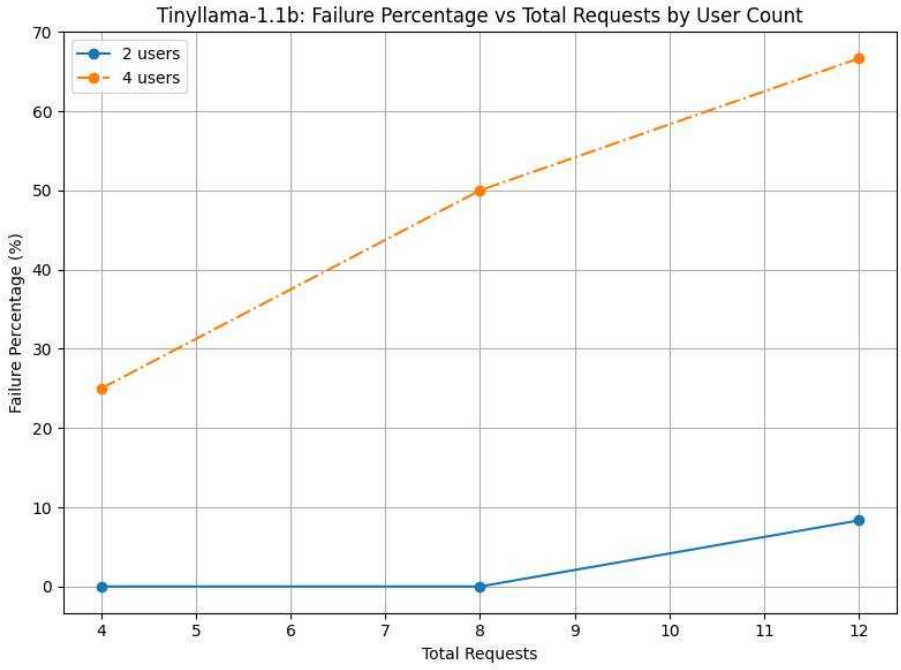


Fig. 13: TinyLlama-1.1B Failure vs Requests

CPU threads more efficiently. The system failed to generate any output because larger models took too long to respond which resulted in zero effective throughput. (Refer to Figures: Figure 16)

- **Prompt Execution Behavior**: The educational prompt asked students to develop their reasoning abilities by creating written explanations through their assignments. The **TinyLlama-1.1B** model successfully finished all 2-user prompts within the 60-second time frame but **Qwen2.5-3B** reached its timeout threshold when handling more than 4 users. The system memory became fully utilized by large models which resulted in their instant breakdown.
- **Scalability Observation**: The system maintained partial functionality when 4 users accessed the system simultaneously while sending 12 requests to both TinyLlama and Qwen2.5 which demonstrated that edge-based AI tutoring can use small models with quantization at 3B parameters or less.

5.3 Evaluation

Based on the results, the following evaluations were made:

- The **TinyLlama-1.1B** model operates as the most efficient and dependable option for delivering lightweight tutoring responses through factual explana-

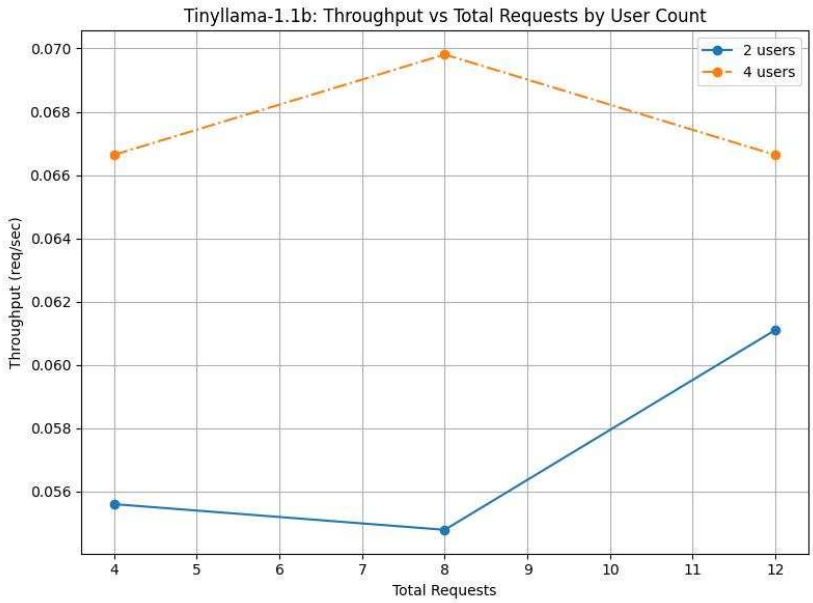


Fig. 14: TinyLlama-1.1B Throughput vs Requests

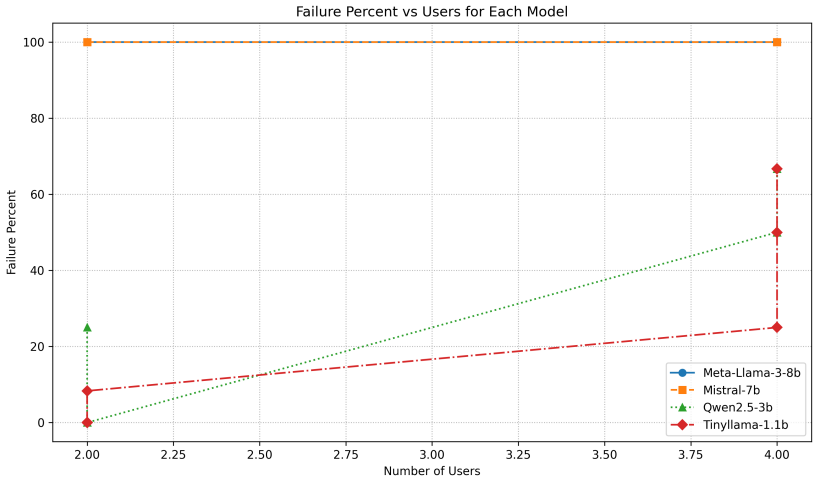


Fig. 15: Failure Rate Comparison

tions and science concepts and short essays. It sustains performance even as the number of concurrent users increases.

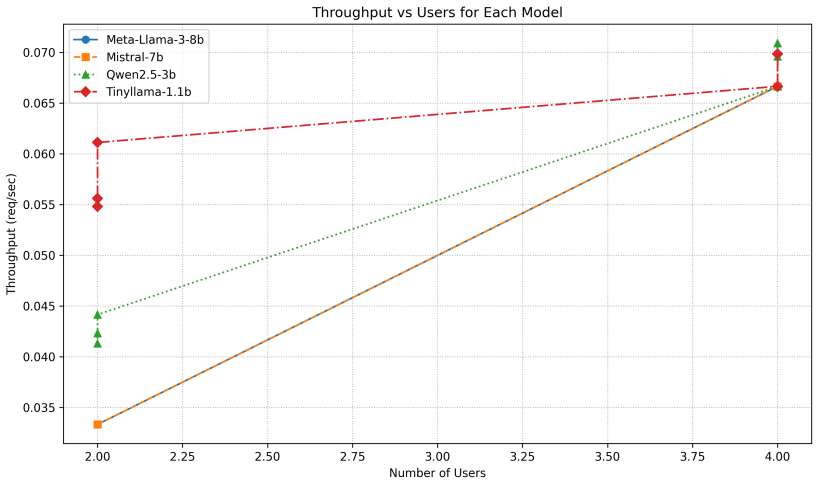


Fig. 16: Throughput Comparison

- The **Qwen2.5-3B** system delivers advanced linguistic capabilities with strong reasoning abilities which work well for individual study needs but its performance deteriorates significantly when multiple users access the system.
- The advanced models **Mistral-7B** and **Meta-Llama-3-8B** require external acceleration or distributed computation support to run on Raspberry Pi 5.

The evaluation results show that **TinyLlama** and **Qwen2.5** offer the best trade-off between accuracy and low latency and easy deployment for educational agents that need to function offline. The **TinyLlama-1.1B** system provides rural classrooms with low power and no internet access a functional method to achieve real-time AI tutoring at their location.

6 Conclusion

The research proves that a **Personalized Offline-Capable AI Tutoring Agent** can run independently on a **Raspberry Pi 5** through the use of lightweight large language models (LLMs). The prototype system which combines Flask-based backend with browser-accessible tutoring interface and quantized models through *llama.cpp* demonstrates that AI tutoring with personalization can reach rural areas and low-resource settings without needing internet access or cloud infrastructure. The experimental results show that **Qwen2.5-3B** operates correctly with acceptable delay times and some degree of task overlap but **TinyLlama-1.1B** operates reliably with low power consumption for real-time applications. The performance of **Mistral-7B** and **Meta-Llama-3-8B** models exceeded hardware capabilities because they demonstrated how model dimensions affect both response speed and power consumption.

The research adds value to technical and educational fields through its development of an operational system for AI tutoring without internet access and its measurement of performance limits on edge devices and its analysis of power consumption and user experience factors. It emphasizes the potential of **edge computing and AI democratization** for rural education, reducing dependency on high-bandwidth internet and centralized cloud services.

However, several **limitations** remain. The current prototype was evaluated in a controlled environment and lacks user testing with rural students and teachers. The system operates with English language instruction only because it fails to support Myanmar and other local languages because of quantized model tokenization and hallucination problems. The system fails to deliver improved personalization and knowledge retention because it does not include GPU acceleration and multilingual datasets and advanced reinforcement learning mechanisms.

The system needs to be tested in real-world settings through field deployment while users in rural educational environments must evaluate its effectiveness for students and teachers and its flexibility across different cultural settings. The system would benefit from additional features which include multilingual support and speech recognition and adaptive learning feedback mechanisms based on reinforcement learning. The system would achieve better sustainability through its integration with renewable energy sources and low-power optimization frameworks. The combination of local inference with opportunistic cloud synchronization through hybrid architectures would provide an optimal solution to achieve both autonomy and performance.

The research creates essential guidelines to enable AI education accessibility through affordable systems which use minimal power and operate independently from the internet to provide digital learning opportunities for underserved populations.

7 APPENDIX

The complete demo implementation and scripts are accessible at GitHub Repository: https://github.com/magicmyo/offline_ai_tutor

config.json

```
{
  "api_url": "http://127.0.0.1:8081/v1/chat/completions",
  "model": "local-model",
  "timeout": 120,
  "web_port" : 8082,
  "system": "You are an offline AI tutoring AGENT. You can
     either answer normally,\nor call ONE tool when useful.
     To call a tool, output EXACTLY one line:\n<tool_call>{\"
     name\":\"TOOL_NAME\",\"arguments\":{...}}</tool_call>\n\
     nAvailable tools:\n1) calculator(expression: str) #
     calculates numeric operations\n2) search_notes(query:
     str) # searches local ./notes. output should return file
      name also\n\nMANDATORY: If the user's message is a math
      expression containing only\ndigits, spaces, and + - * /
      ( ), you MUST call the calculator tool with\nthe exact
     expression. Do NOT compute it yourself. Output only the\
     n<tool_call> line and nothing else.\n\nRULES:\n- If the
     user's message CONTAINS any arithmetic expression (
     digits and + - * / ( )),\n  EXTRACT the expression (e.g
     ., \"8/2\", \"(2+3)*4\") and call the calculator with it
     .\n  Do NOT compute yourself. Output only the <tool_call
     >.\n- Use search_notes only when the user asks to find/
     read notes or mentions chapters/pages\n  without a
     solvable expression.\n\nExamples:\nUser: \"Simplify 8/2
     from Chapter 3 fractions\" \n<tool_call>{\"name\":\"
     calculator\",\"arguments\":{\"expression\":\"8/2\"}}</
     tool_call>\n\nUser: \"Find Chapter 3 fractions\"\n<
     tool_call>{\"name\":\"search_notes\",\"arguments\":{\"
     query\":\"Chapter 3 fractions\"}}</tool_call>\n\nAfter
     the tool returns, write a clear final answer using the
     tool result.\nKeep outputs concise in less than 100
     words and helpful for students.",
  "subject_primers": {
    "Coding":  "You are tutoring Coding. Answer in under 100
       words. Prefer step-by-step explanation with tiny
       examples ( 15  lines).",
    "Math":    "You are tutoring Math. Answer in under 100
       words. Use concrete objects. Keep reasoning short.",
    "Science": "You are tutoring Science. Answer in under 100
       words. Use everyday phenomena.",
    "English": "You are tutoring English. Answer in under 100
       words. Teach with short examples, then give one tiny
       practice."
  }
```

```
}
```

index.html

```
<!doctype html>
<html lang="en">
    <head>
        <meta charset="utf-8"/>
        <meta content="width=device-width,initial-scale=1"
            name="viewport"/>
        <title>       AI Tutor</title>

        <script defer src="js/script.js?v=2"></script>
        <link rel="stylesheet" href="css/styles.css?v=2">
    </head>

    <body>

        <div class="app">
            <div class="title">AI Tutor     Offline & Light</
                div>
            <div class="tabs" id="tabs"></div>

            <div class="hint">Use <b>Agent Mode</b> tab for
                tool calls (e.g., search my notes for
                photosynthesis , (2+3)*4 ).
                Other
                tabs act as normal chat.
            </div>
            <div class="chat">
                <div class="msgs" id="msgs"></div>
                <div class="row">
                    <input id="inp" placeholder="Type your
                        question "/>
                    <button class="btn" id="send">Send</button
                        >
                </div>
            </div>

            <div class="muted" style="margin-top:8px">
                Tip: Connect to Pi Access Point hotspot first
                    to use offline AI Tutor.
            </div>
        </div>
    </body>
</html>
```

agent_server.py

```
import os, re, json, glob, requests
from typing import Dict, List, Optional
```

```
from flask import Flask, request, jsonify, send_from_directory
    , make_response
from pypdf import PdfReader

CONFIG_PATH = os.environ.get("WEB_CONFIG", "./config.json")
_CONFIG_MTIME = None

# defaults (fallbacks if config.json is missing)
DEFAULT_SYSTEM = """
You are an offline AI tutoring AGENT. You can either answer
    normally,
or call ONE tool when useful. To call a tool, output EXACTLY
    one line:
<tool_call>{"name":"TOOL_NAME","arguments":{...}}</tool_call>

Available tools:
1) calculator(expression: str) #calculates numeric operations
2) search_notes(query: str) # searches local ./notes. output
    should return file name also

MANDATORY: If the user's message is a math expression
    containing only
digits, spaces, and + - * / ( ), you MUST call the calculator
    tool with
the exact expression. Do NOT compute it yourself. Output only
    the
<tool_call> line and nothing else.

RULES:
- If the user's message CONTAINS any arithmetic expression (
    digits and + - * / ( )),
  EXTRACT the expression (e.g., "8/2", "(2+3)*4") and call the
      calculator with it.
  Do NOT compute yourself. Output only the <tool_call>.
- Use search_notes only when the user asks to find/read notes
    or mentions chapters/pages
  without a solvable expression.

Examples:
User: "Simplify 8/2 from Chapter 3 fractions"
<tool_call>{"name":"calculator","arguments":{"expression
    ":"8/2"}}</tool_call>

User: "Find Chapter 3 fractions"
<tool_call>{"name":"search_notes","arguments":{"query":"
    Chapter 3 fractions"}}</tool_call>

After the tool returns, write a clear final answer using the
    tool result.
```

```
    Keep outputs concise in less than 100 words and helpful for
        students.
    """

    # behaviour by Subjects
    DEFAULT_SUBJECT_PRIMERS = {
        "Coding":  "You are tutoring Coding. Answer in under 100
            words. Prefer step-by-step explanation with tiny
            examples ( 15  lines).",
        "Math":    "You are tutoring Math. Answer in under 100
            words. Use concrete objects. Keep reasoning short.",
        "Science": "You are tutoring Science. Answer in under 100
            words. Use everyday phenomena.",
        "English": "You are tutoring English. Answer in under 100
            words. Teach with short examples, then give one tiny
            practice.",

    }


    def _apply_config(conf: dict):
        global API_URL, MODEL, TIMEOUT, SYSTEM, SUBJECT_PRIMERS,
            WEB_PORT
        API_URL = conf.get("api_url", os.environ.get("LLAMA_URL",
            "http://127.0.0.1:8081/v1/chat/completions"))
        MODEL   = conf.get("model",   os.environ.get("LLAMA_MODEL"
            , "local-model"))
        TIMEOUT = int(conf.get("timeout", 120))
        SYSTEM  = conf.get("system", DEFAULT_SYSTEM)
        WEB_PORT = int(conf.get("web_port", 8000))
        SUBJECT_PRIMERS = conf.get("subject_primers",
            DEFAULT_SUBJECT_PRIMERS)

    def _load_config_file() -> dict:
        try:
            with open(CONFIG_PATH, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            return {}

    def maybe_reload_config():
        """Reload config.json when it changes (cheap, called per
            request)."""
        global _CONFIG_MTIME
        try:
            m = os.path.getmtime(CONFIG_PATH)
            if _CONFIG_MTIME != m:
                _apply_config(_load_config_file())
                _CONFIG_MTIME = m
        except Exception:
```

```
        if _CONFIG_MTIME is None:
            # first run and no file     still apply defaults
            _apply_config({})
            _CONFIG_MTIME = 0

# initial load
maybe_reload_config()

# Tools
def calculator(expression: str) -> str:

    try:
        if not re.fullmatch(r"[\d()+\-*/.\s]+", expression):
            return "Blocked: only numbers and + - * / ( )
                allowed."
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
        return f"Calculator error: {e}"

def search_notes(query: str, base_dir: str = "./notes") -> str
    :
    os.makedirs(base_dir, exist_ok=True)
    hits = []
    for path in glob.glob(os.path.join(base_dir, "**", "*.*"),
         recursive=True):
        low = path.lower()
        if low.endswith((".txt", ".md")):
            try:
                with open(path, "r", encoding="utf-8", errors=
                    "ignore") as f:
                    text = f.read()
                if query.lower() in text.lower():
                    i = text.lower().find(query.lower())
                    s = max(0, i - 120); e = min(len(text), i
                        + 120)
                    hits.append({"file": path, "snippet": text
                        [s:e].replace("\n"," ")})
            except:
                pass
        elif low.endswith(".pdf"):
            try:
                pdf = PdfReader(path)
                first_pages = "\n".join([(p.extract_text() or
                    "") for p in pdf.pages[:3]])
                if query.lower() in first_pages.lower():
                    i = first_pages.lower().find(query.lower()
                        )
                    s = max(0, i - 120); e = min(len(
                        first_pages), i + 120)
```

```
                    hits.append({"file": path, "snippet":
                        first_pages[s:e].replace("\n"," ")})
            except:
                pass
    if not hits:
        return "No matches."
    return "\n".join([f"- {h['file']}: {h['snippet']}" for h
        in hits[:8]])

TOOLS = {
    "calculator": {
        "fn": calculator,
        "schema": {"type":"object","properties":{"expression"
            :{"type":"string"}},"required":["expression"]}
    },
    "search_notes": {
        "fn": search_notes,
        "schema": {"type":"object","properties":{"query":{"
            type":"string"}},"required":["query"]}
    }
}



def build_agent_prompt(user_text: str) -> List[Dict[str,str]]:
    """Agent mode: pure SYSTEM rules, no tutoring primer."""
    return [
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": user_text}
    ]


def llm(messages: List[Dict[str,str]]) -> str:
    payload = {"model": MODEL, "messages": messages, "
        temperature": 0.2, "max_tokens": 200, "stream": False}
    r = requests.post(API_URL, json=payload, timeout=TIMEOUT)
    r.raise_for_status()
    data = r.json()
    return data["choices"][0]["message"]["content"]

TOOL_RE = re.compile(r"<tool_call>\s*(\{.*?})\s*</tool_call>",
     re.DOTALL)

def extract_tool_json(text: str) -> Optional[str]:
    # try strict match first
    m = TOOL_RE.search(text)
    if m:
        return m.group(1)
```

```
    # fallback: handle missing </tool_call>; scan balanced
        braces
    i = text.find("<tool_call>")
    if i == -1:
        return None
    j = text.find("{", i)
    if j == -1:
        return None
    depth = 0
    for k, ch in enumerate(text[j:], start=j):
        if ch == "{":
            depth += 1
        elif ch == "}":
            depth -= 1
            if depth == 0:
                return text[j:k+1]
    return None

def agent_answer(user_text: str, subject: Optional[str] = None
    ) -> Dict[str, Optional[str]]:
    maybe_reload_config()

    if subject == "Agent Mode":
        messages = build_agent_prompt(user_text)
    else:
        subj_sysprompt = SUBJECT_PRIMERS.get(subject, "")
        sysprompt = subj_sysprompt
        messages = [{"role": "system", "content": sysprompt},
            {"role": "user", "content": user_text}]

    out = llm(messages)

    spec_text = extract_tool_json(out.strip())
    if not spec_text:
        return {"reply": out, "tool": None, "tool_result":
            None}

    tool = None; tool_result = None
    try:
        #spec = json.loads(m.group(1))
        spec = json.loads(spec_text)
        name = spec.get("name", "")
        args = spec.get("arguments", {})
        tool = name
        if name not in TOOLS:
            final = f"I attempted to use an unavailable tool
                '{name}'."
        else:
            fn = TOOLS[name]["fn"]
            tool_result = fn(**args)
```

```
            # feed tool result + subject primer again so the
                final answer stays on style
            messages.append({"role": "assistant", "content":
                out})
            messages.append({"role": "system", "content": f"
                Tool '{name}' returned:\n{tool_result}"})
            if subject and subject in SUBJECT_PRIMERS:
                messages.append({"role": "system", "content":
                    SUBJECT_PRIMERS[subject]})
            final = llm(messages)
        return {"reply": final, "tool": tool, "tool_result":
            tool_result}
    except Exception as e:
        return {"reply": f"I tried to call a tool but failed:
            {e}", "tool": tool, "tool_result": tool_result}

# Flask app (serves API + index.html)
app = Flask(__name__, static_folder=".", static_url_path="")

def _cors(resp):
    resp.headers["Access-Control-Allow-Origin"] = "*"
    resp.headers["Access-Control-Allow-Headers"] = "Content-
        Type"
    resp.headers["Access-Control-Allow-Methods"] = "POST, GET,
         OPTIONS"
    return resp

@app.route("/", methods=["GET"])
def root():
    return send_from_directory(".", "index.html")

@app.route("/chat", methods=["POST", "OPTIONS"])
def chat():
    if request.method == "OPTIONS":
        return _cors(make_response())
    data = request.get_json(force=True, silent=True) or {}
    msg = (data.get("message") or "").strip()
    subject = (data.get("subject") or "").strip()
    if not msg:
        return _cors(jsonify({"error": "Empty message"})), 400
    result = agent_answer(msg, subject=subject or None)
    return _cors(jsonify(result))

@app.route("/config", methods=["GET"])
def get_config():
    maybe_reload_config()
    data = {
        "api_url": API_URL,
        "model": MODEL,
        "timeout": TIMEOUT,
```

```
        "subject_primers": SUBJECT_PRIMERS,
        "subjects": list(SUBJECT_PRIMERS.keys()) + ["Agent
            Mode"]
    }
    return _cors(jsonify(data))

@app.route("/favicon.ico")
def favicon():
    return send_from_directory(".", "images/favicon.ico",
        mimetype="image/x-icon")

if __name__ == "__main__":
    port = WEB_PORT #int(os.environ.get("PORT", "8000"))
    app.run(host="0.0.0.0", port=port, debug=False)
```

tpLink.sh

```
#!/usr/bin/env bash
set -e

sudo systemctl stop dnsmasq


# ===== CONFIGURATION =====
SSID="AITutor"
PASSWORD="pass12345"
WIFI_IF="wlan1"
INTERNET_IF="eth0"    # or "" for no internet sharing
SECURITY="WPA2"       # WPA, WEP, or nopass

# ===== DISCONNECT EXISTING WI-FI =====
echo "[..] Disconnecting any existing Wi-Fi connection..."
sudo nmcli dev disconnect wlan0 2>/dev/null || true
sudo ip link set wlan0 down && sudo ip link set "$WIFI_IF" up

sudo nmcli dev disconnect "$WIFI_IF" 2>/dev/null || true
sudo ip link set "$WIFI_IF" down && sudo ip link set "$WIFI_IF
    " up

clear

echo "========================================="
echo "   Raspberry Pi Access Point TPLink Setup"
echo "========================================="
echo " - Wi-Fi Name (SSID): $SSID"
echo " - Password: $PASSWORD"
echo " - Security: $SECURITY"
```

```
if [ -n "$INTERNET_IF" ]; then
    echo " - Internet Sharing: Enabled via $INTERNET_IF"
else
    echo " - Internet Sharing: Disabled (Offline LAN mode)"
fi
echo " - Current IP Address(es): $(hostname -I)"
echo "========================================="
sleep 2

echo

sudo rfkill unblock all
sudo nmcli dev set wlan1 managed no

# ===== SHOW QR CODE IN TERMINAL =====
qrencode -t ANSIUTF8 "WIFI:T:$SECURITY;S:$SSID;P:$PASSWORD;;"

echo



# ===== START ACCESS POINT =====
sudo create_ap --no-virt -n "$WIFI_IF" "$SSID" "$PASSWORD" --
    freq-band 2.4 --country NZ --ieee80211n
```

agent_ui.sh

```
#!/usr/bin/env bash
set -euo pipefail

#### ==== CONFIG ==== ####
MODEL_PATH="/home/offline_ai_tutor/models/qwen2.5-3b-instruct-
    q4_k_m.gguf"
LLAMA_PORT=8081              # llama-server API port
WEB_PORT=8082                # index.html port
AGENT_PATH="/home/offline_ai_tutor/ai_tutor_ui/"
IFACE="${IFACE:-}"           # optionally export IFACE=wlan0
    or eth0
OPEN_BROWSER="${OPEN_BROWSER:-0}"  # set to 1 to xdg-open the
    UI
QR_OUT="${QR_OUT:-qr_ui_port_${WEB_PORT}.png}"

#### ==== helpers: IP detection & QR ==== ####
ip_of() {
  local dev="$1"
  ip -4 addr show "$dev" 2>/dev/null \
    | awk '/inet /{print $2}' | cut -d/ -f1 | grep -v '^127\.'
        || true
}
```

```
pick_ip() {
  local ip=""
  [[ -n "$IFACE" ]] && ip="$(ip_of "$IFACE")"
  [[ -z "$ip" ]] && ip="$(ip_of wlan0)"
  [[ -z "$ip" ]] && ip="$(ip_of eth0)"
  if [[ -z "$ip" ]]; then
    for i in $(hostname -I 2>/dev/null || true); do
      if echo "$i" | grep -q '^192\.168\.'; then
        ip="$i"; break
      fi
    done
  fi
  echo "$ip"
}

need_qrencode() {
  if ! command -v qrencode >/dev/null 2>&1; then
    echo "[..] 'qrencode' not found. Attempting to install..."
    if command -v apt >/dev/null 2>&1; then
      sudo apt update && sudo apt install -y qrencode
    else
      echo "[!!] Please install 'qrencode' manually and re-run
          ."
      exit 1
    fi
  fi
}

#### ==== start servers ==== ####
echo "[..] Starting llama-server on port $LLAMA_PORT ..."
llama-server \
  -m "$MODEL_PATH" \
  --host 0.0.0.0 \
  --port $LLAMA_PORT \
  -c 16384 \
  -np 4 \
  >/tmp/llama-server.log 2>&1 &
LLAMA_PID=$!

echo "[..] Starting static web server on port $WEB_PORT ..."
# python3 -m http.server $WEB_PORT --bind 0.0.0.0 --directory
    "$(dirname "$INDEX_HTML")"
cd "$AGENT_PATH"
python3 agent_server.py \
  >/tmp/agent-ui-web.log 2>&1 &
WEB_PID=$!

trap 'echo; echo "[..] Stopping..."; kill $LLAMA_PID $WEB_PID
    2>/dev/null || true' INT TERM
```

```
# small warm-up
sleep 1

#### ==== print endpoints + QR ==== ####
IP="$(pick_ip)"
if [[ -z "$IP" ]]; then
  echo "[!!] Could not detect a LAN IP. Are you connected to a
      network?"
  kill $LLAMA_PID $WEB_PID 2>/dev/null || true
  exit 1
fi

UI_URL="http://${IP}:${WEB_PORT}"     # index.html is served
    as directory index
API_URL="http://${IP}:${LLAMA_PORT}"   # llama-server root;
    add /v1/chat/completions in your UI

clear

echo
echo "================= ENDPOINTS ================="
echo " Hostname       : $(hostname)"
echo " All IPs        : $(hostname -I || true)"
echo " Selected IP    : ${IP}"
echo " UI (index.html): ${UI_URL}"
echo " API (llama)    : ${API_URL}"
echo " Logs           : /tmp/llama-server.log  |  /tmp/tutor-
    ui-web.log"
echo "============================================="
echo

need_qrencode
echo "[..] UI QR code (scan with phone):"
qrencode -t ANSIUTF8 "$UI_URL"

if [[ "$OPEN_BROWSER" == "1" ]] && command -v xdg-open >/dev/
    null 2>&1; then
  xdg-open "$UI_URL" >/dev/null 2>&1 || true
fi

# keep both services alive
wait
```

References

1. Alhafni, B., Vajjala, S., Bannò, S., Kumar, M. K., & Kochmar, E. (2024). *LLMs in Education: Novel Perspectives, Challenges, and Opportunities*. ArXiv.org. https://arxiv.org/abs/2409.11917
2. Wen, Q., Liang, J., Sierra, C., Luckin, R., Tong, R., Liu, Z., Cui, P., & Tang, J. (2024). *AI for Education (AI4EDU): Advancing Personalized Education with LLM and Adaptive Learning*. 6743–6744. https://doi.org/10.1145/3637528.3671498
3. Choi, J. H., Garrod, O., Atherton, P., Joyce-Gibbons, A., Mason-Sesay, M., & Björkegren, D. (2023). *Are LLMs Useful in the Poorest Schools? TheTeacher.AI in Sierra Leone*. ArXiv.org. https://arxiv.org/abs/2310.02982
4. Denny, P., MacNeil, S., Jaromir Savelka, Porter, L., & Luxton-Reilly, A. (2024). Desirable Characteristics for AI Teaching Assistants in Programming Education. *ArXiv (Cornell University)*. https://doi.org/10.1145/3649217.3653574
5. Zhu, T., Zhang, K., & Wang, W. Y. (2024). *Embracing AI in Education: Understanding the Surge in Large Language Model Use by Secondary Students*. ArXiv.org. https://arxiv.org/abs/2411.18708
6. Kamalov, F., Calonge, D. S., Smail, L., Azizov, D., Thadani, D. R., Kwong, T., & Atif, A. (2025). *Evolution of AI in Education: Agentic Workflows*. ArXiv.org. https://arxiv.org/abs/2504.20082
7. Ferrag, M. A., Tihanyi, N., & Debbah, M. (2025). *From LLM Reasoning to Autonomous AI Agents: A Comprehensive Review*. ArXiv.org. https://arxiv.org/abs/2504.19678
8. Jaldi, C. D., Ilkou, E., Schroeder, N., & Shimizu, C. (2024). Education in the era of Neurosymbolic AI. *Journal of Web Semantics*, *85*, 100857. https://doi.org/10.1016/j.websem.2024.100857
9. Jin, H., Lee, S., Shin, H., & Kim, J. (2024). *Teach AI How to Code: Using Large Language Models as Teachable Agents for Programming Education*. https://doi.org/10.1145/3613904.3642349
10. Bali, B. (2025). *A Novel Context-Aware Intelligent Learning Framework for Personalized Education in Developing Countries*. https://doi.org/10.2139/ssrn.5357067
11. Karimy, A. U., Rasuli, J., Reddy, P. C., Joya, M., Hamdard, A. J., & Ghulami, H. R. (2024). A Review on the Feasibility of AI-Supported Education Platforms in Afghanistan: Addressing Barriers to Women and Girls' Education. *2024 IEEE Global Humanitarian Technology Conference (GHTC)*, 1–8. https://doi.org/10.1109/ghtc62424.2024.10771581
12. Karimy, A. U., Rasuli, J., & Reddy, P. C. (2024). AI in Online Educational Access: Breaking Barriers for Female Learners. *2024 21st International Conference on Information Technology Based Higher Education and Training (ITHET)*, 1–9. https://doi.org/10.1109/ithet61869.2024.10837681
13. Ghorbani, A., & Fattahi, H. (2025). *Bridging the Digital Divide: Small Language Models as a Pathway for Physics and Photonics Education in Underdeveloped Regions*. ArXiv.org. https://arxiv.org/abs/2506.12403
14. Trajkovski, G., & Hayes, H. (2025). Implementing AI-Assisted Assessment in Educational Institutions. *Digital Education and Learning*, 193–243. https://doi.org/10.1007/978-3-031-88252-4_5
15. Leong, W. Y. (2024). Transforming Rural and Underserved Schools with AI-Powered Education Solutions. *ASM Science Journal*, *19*, 1–12. https://doi.org/10.32802/asmscj.2023.1895

16. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems*, *6*, 87–100.
17. Yi, R., Guo, L., Wei, S., Zhou, A., Wang, S., & Xu, M. (2025). EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices. *IEEE Transactions on Mobile Computing*, *24*(8), 7059–7073. https://doi.org/10.1109/tmc.2025.3546466
18. Ardakani, M., Malekar, J., & Zand, R. (2025). LLMPi: Optimizing LLMs for High-Throughput on Raspberry Pi. *Thecvf.com*, 6378–6387.
19. Sakai, K., Uehara, Y., & Kashihara, S. (2024). Implementation and Evaluation of LLM-Based Conversational Systems on a Low-Cost Device. *2024 IEEE Global Humanitarian Technology Conference (GHTC)*, 392–399. https://doi.org/10.1109/ghtc62424.2024.10771565
20. Nguyen, V., Dhopate, V., Huynh, H., Bouhlal, H., Annengala, A., Scoccia, G. L., Martinez, M., Stoico, V., & Malavolta, I. (2025). On-Device or Remote? On the Energy Efficiency of Fetching LLM-Generated Content. *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 72–82. https://doi.org/10.1109/cain66642.2025.00016
21. Islam, Md. M., Funabiki, N., Kuribayashi, M., Debnath, S. K., Munene, K. I., Lwin, K. S., Sudibyo, R. W., & Mamun, Md. S. A. (2018). Dynamic Access-Point Configuration Approach for Elastic Wireless Local-Area Network System and Its Implementation Using Raspberry Pi. *International Journal of Networking and Computing*, *8*(2), 254–281. https://doi.org/10.15803/ijnc.8.2_254
22. Cheah, W., Zhao, Jegatheesan, J., & Loon, S. (2015). Exploring IOT Application Using Raspberry Pi. *International Journal of Computer Networks and Applications*, *2*(1), 27. https://ijcna.org/Manuscripts/Volume-2/Issue-1/Vol-2-issue-1-M-04.pdf
23. Slavin, A., Norbisrath, U., Morgan, D., & Vainikko, E. (2024). Investigating Raspberry Pi Access Point Client Limit for Use in IoT Education. *2024 11th International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*, 232–239. https://doi.org/10.1109/iotsms62296.2024.10710248
24. Jadhav, A. P., & Malode, V. B. (2019). *Raspberry PI Based OFFLINE MEDIA SERVER*. https://doi.org/10.1109/iccmc.2019.8819718
25. Pierson, D. S. (2016). *Raspberry Pi VPN Travel Router*. DigitalCommons@CalPoly. https://digitalcommons.calpoly.edu/cscsp/83/